Authorization Code Flow with Proof Key for Code Exchange (PKCE)

less than a 15 minute read

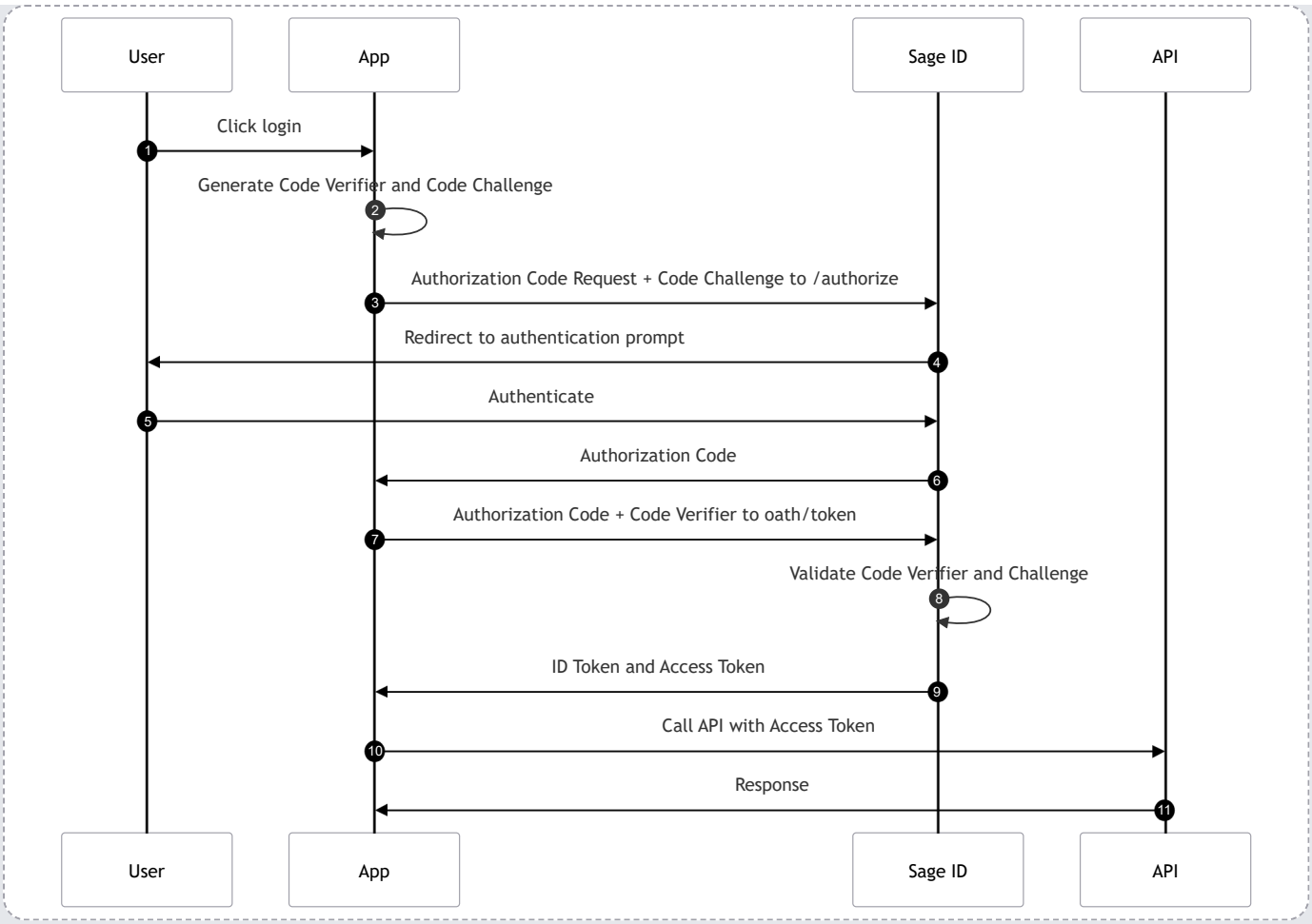
On this page

Overview

Authorization Code using Proof Key for Code Exchange (PKCE) is an **extension** of the **Authorization code grant** flow. It is used for **applications that cannot store a client secret securely**, such as native or single-page apps. If PKCE is not used with a non-confidential client the `authorization_code` could be **intercepted** and exchanged for an **Access Token** (and possibly a Refresh Token).

> **Warning**
>
> Sage ID doesn't support the Implicit grant.

> **Warning**
>
> PKCE must be used with non-confidential clients.



1. User selects Login within application.

2. Create a cryptographically-random `code_verifier` and from this generate a `code_challenge` request using SHA256.

3. Redirect user to Sage ID (`/authorize` endpoint) with the `code_challenge`.

4. Redirect user to login and **authentication** prompt.

5. The user authenticates.

6. Sage ID stores the `code_challenge` and **redirects** to the native app with an **Authorization Code**.

7. The native app sends the `authorization_code` and `code_verifier` with the `redirect_uri` and the `client_id` to Sage ID (/oauth/token endpoint).

8. Sage ID **verifies** the `code_challenge` and `code_verifier`.

9. Sage ID responds with an **ID Token** and **Access Token** (and optionally, a **Refresh Token**).

10. Application can use the Access Token to call the API.

11. API responds with requested data.

> In OAuth 2.0 terms, the web app is the application, the end user the Resource Owner, the API the Resource Server, and Sage ID the Authorization Server.

Pre-requisites

Your **API** and **client applications must be correctly configured** in Sage ID using **Developer Self Service**.

The application owner needs to perform the following configurations:

1. Select **Mobile Application** or **Single Page Application** as the client type. Choose Mobile Application if you have a **Desktop App**.
2. Make sure that your application **grant types** include **Authorization Code**.
3. If you want to be able to use **refresh tokens**, make sure the Application's **grant types** include refresh token. More information on Tokens is available in the Tokens section.
4. Provide a **callback URL**.

The API owner needs to perform the following configurations:

1. Enable `offline_access` if refresh tokens are needed.

> **Warning**
>
> Rotating refresh tokens are required.

How to Implement PKCE Grant

1. Create a Code Verifier

Create a `code_verifier`, which is a cryptographically-random, Base64-encoded key.

javascript Copy

```javascript
function base64URLEncode(str) {
  return str.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=/g, '');
}
var verifier = base64URLEncode(crypto.randomBytes(32));
```

2. Create a Code Challenge

Generate a `code_challenge` from the `code_verifier` that will be sent in the **authorization request**.

> **You must hash this value and not send a plain-text string.**

javascript Copy

```javascript
function sha256(buffer) {
  return crypto.createHash('sha256').update(buffer).digest();
}
var challenge = base64URLEncode(sha256(verifier));
```

3. Call the Authorization URL

Send the user to the **authorization URL**, including the `code_challenge` and the method used to generate it:

txt Copy

```txt
https://id.sage.com/authorize?
audience=API_AUDIENCE&
scope=SCOPE&
response_type=code&
client_id=YOUR_CLIENT_ID&
code_challenge=CODE_CHALLENGE&
code_challenge_method=S256&
redirect_uri=https://YOUR_APP/callback
```

Parameter	Description
`audience`	**Unique identifier** of the API, only needed for Access Tokens.
`scope`	The scopes that the authorization is requested for. Must be separated by a space. Any standard OIDC scopes can be requested (such as `profile` and `email`), custom claims, or any scopes supported by the API. Include `offline_access` to get a Refresh Token.
`response_type`	The credential that Sage ID will return. For this flow, the value **must** be **code**.
`client_id`	The application's Client ID.
`state`	An opaque value the application adds to the initial request that Sage ID includes when redirecting back to the application. This value **must be used** by the application **to prevent CSRF**(Cross Site Request Forgery) **attacks**.
`redirect_uri`	The URL to where the browser will be **redirected after authentication** of the user. The `Authorization Code` will be available in the code URL parameter. This URL must be specified as a valid callback URL when you provide the API configuration.
`login_hint`	An `email address` or a base64 and URL encoded **JSON object** may be provided that can be pre-populated on the authentication screen. See Login hints.
`code_challenge`	Generated challenge from the `code_verifier` using SHA256.
`code_challenge_method`	Method used to generate the challenge. The only accepted value is 'S256'.

html Copy

```
<a href="https://id-shadow.sage.com/authorize?scope=appointments%20contacts&audience=app
  Sign In
</a>
```

The authorization code is included in the query string parameter after `code=` :

html Copy

```html
HTTP/1.1 302 Found
Location: {https://YOUR_APP/callback&}?code={authorizationCode}&state=xyzABC123
```

4. Exchange the Authorization Code for a Token

Using the Authorization Code, cerate a **POST** request to the Token URL with the `code_verifier` :

c# Copy

```csharp
var client = new RestClient("https://id.sage.com/oauth/token");
var request = new RestRequest(Method.POST);
request.AddHeader("content-type", "application/x-www-form-urlencoded");
request.AddParameter("application/x-www-form-urlencoded", "grant_type=authorization_code
IRestResponse response = client.Execute(request);
```

Where:

- `grant_type` : This must be `authorization_code`
- `client_id` : Your application's Client ID.
- `code_verifier` : Cryptographically random key that was used to generate the `code_challenge` passed to `/authorize` .
- `code` : The Authorization Code.
- `redirect_url` : The URL must match exactly the `redirect_uri` passed to `/authorize`

Each parameter must be URL encoded according to RFC1866 section 8.2.1 so spaces must be encoded as a + (plus) character.

The response contains `access_token` , `refresh_token` , `id_token` , and `token_type` values, for example:

javascript Copy

```javascript
{
  "access_token": "eyJz93a...k4laUWw",
  "refresh_token": "GEbRxBN...edjnXbL",
  "id_token": "eyJ0XAi...4faeEoQ",
  "token_type": "Bearer"
}
```

The `refresh_token` will only be present in the response if you included the `offline_access` scope and your application is configured to **Allow Offline Access** in Sage ID.

5. Call the API

Calls can be made to the API by passing the `access_token` as a Bearer Token in the `Authorization` header:

javascript Copy

```javascript
// Use the Access Token to make API calls
  $('#get-appointments').click(function(e) {
    e.preventDefault();

    $.ajax({
      cache: false,
      url: "http://localhost:7001/api/appointments",
      headers: { "Authorization": "Bearer " + access_token }
    });
  });
```

See how to **Verify the token**.

Previous: Authorization Code Grant

Was this page helpful?

Yes No

Report issue